

Phase 5 - Project Development

PocketSmart AI: Your Smart Budget & Recommendation Assistant

Date: 29 September 2026

1. Development Overview

PocketSmart AI is developed as a modular web application using FastAPI, Gemini 1.5 Flash Pro, HTML/CSS + JavaScript, Jinja2 templates, authentication/session handling and a database layer.

2. Environment Setup

- Prepare Python environment.
- Install requirements.txt dependencies.
- Create .env from .env.example.
- Configure required AI credentials securely.
- Prepare the database/data directory.

3. AI Initialization

Development begins by validating Gemini access and testing text and image + text prompts. Prompts are refined for budget interpretation and recommendation generation.

Project Development - Backend and Planner Routes

4. FastAPI Initialization

- Import required libraries.
- Initialize FastAPI.
- Load .env configuration.
- Register routers and services.

5. Core Routes

Route	Function
/generate-home	Home furniture, décor and lighting recommendations
/generate-party	Party catering, venue and decoration recommendations
/generate-jewelry	Text + optional image jewelry recommendations

6. Recommendation Logic

The recommendation service prepares user context and coordinates AI calls and product/platform information. Conditional logic can categorize situation-based recommendations.

Project Development - Authentication and Frontend

7. Authentication

- /register – new user registration.
- /login – authentication and session start.
- /logout – session termination.
- /token – JWT token issuance.
- /session-info – session metadata.
- /session-data – session-specific information.

8. Service Layer

- catalog.py – catalog/platform data.
- gemini_utils.py – Gemini communication and AI utilities.
- recommendation_service.py – recommendation processing.

9. Frontend

- HTML/CSS + JavaScript interface.
- Jinja2 templates.
- Planner forms for all three domains.
- Dynamic and readable recommendation results.

Project Development - Persistence and Outcome

10. Database

The project structure includes data/pocketsmart.db, which is used by the database layer for application persistence.

11. Error and Fallback Handling

The project specifies validations and fallback/default recommendations when AI returns insufficient results, helping keep the recommendation workflow usable.

12. Project Structure

The root contains .env, .env.example, .gitignore, requirements.txt, README.md, data, tests and app. The app package contains authentication, configuration, database, models, routers, services, static assets and templates.

13. Development Outcome

The development connects user input, FastAPI routes, recommendation services, Gemini processing, platform-oriented data and frontend result display.